

Neural Network–Assisted Variational Quantum Circuit Pipeline:

From Barren–Plateau Mitigation to
Multi-Mode Quantum Optimisation

PH5393 Project Report

G Madhukar - EP23BTECH11011

Panikar - CS23BTECH11040

Mohit Naidu - CS23BTECH11051



భారతీయ సాంకేతిక విజ్ఞాన సంస్థ హైదరాబాద్
भारतीय प्रौद्योगिकी संस्थान हैदराबाद
Indian Institute of Technology Hyderabad

Dr. Sangkha Borah
Department of Physics
Indian Institute of Technology Hyderabad

Abstract

This report provides a comprehensive exposition of a two-module hybrid quantum-classical software system designed to address the *barren plateau* (BP) problem in Variational Quantum Circuits (VQCs). The first module, `Training.py`, implements gradient-variance benchmarking and Neural Network (NN)-based parameter initialisation for a 6-qubit Variational Quantum Eigensolver (VQE) applied to the Transverse-Field Ising Model (TFIM). The second module, `pipeline.py`, extends this into a production-ready inference system with four distinct pipelines: (1) NN warm-start VQE, (2) Hamiltonian transfer benchmarking, (3) candidate parameter generation and ranking, and (4) multi-format model export.

The barren plateau phenomenon — wherein the variance of cost-function gradients decays exponentially with the number of qubits and circuit depth — represents one of the most fundamental trainability obstacles in near-term quantum computing. This work adopts the mitigation strategy of Friedrich and Maziero (2022), wherein a lightweight two-layer feedforward neural network is trained to generate circuit parameters biased toward high-gradient, low-energy regions of the optimisation landscape, bypassing the need for uniform random initialisation entirely.

Starting from first principles in quantum mechanics and machine learning, the report builds a complete theoretical foundation — covering Hilbert space formalism, unitary gate operations, the variational principle, the parameter-shift rule, and backpropagation through a quantum layer — before dissecting every function, design decision, and algorithmic component of both files. The hardware-efficient ansatz employed consists of alternating R_y - R_z single-qubit rotation blocks and linear CNOT entangling layers, with a total of $M = 2nL$ trainable parameters for n qubits and L circuit layers.

Empirical results confirm that NN-generated initialisation sustains gradient variance at circuit depths where uniform random initialisation fails, and converges to the exact ground state energy of $E_0 = -6.385$ where classical Adam optimisation stalls. Transfer benchmarking across twelve distinct (J, h) Hamiltonian configurations reveals that the depth-4 model generalises more robustly than the depth-10 model to off-training-point coupling regimes. The production pipeline further demonstrates practical utility through ranked candidate generation, multi-format model export (JSON, CSV, PyTorch, and raw VQC parameters), and a fully documented command-line interface suitable for integration into broader quantum chemistry or condensed matter workflows.

Contents

List of Abbreviations	5
1 Introduction and Motivation	6
1.1 The Quantum Computing Landscape	6
1.2 Variational Quantum Algorithms	6
1.3 The Barren Plateau Crisis	6
1.4 The Neural Network Mitigation Strategy	7
1.5 Project Scope and Report Structure	7
2 Quantum Mechanics Foundations	8
2.1 State Vectors and the Hilbert Space	8
2.1.1 Multi-Qubit States and Tensor Products	8
2.2 Quantum Gates and Unitary Evolution	8
2.2.1 Pauli Matrices	9
2.2.2 Rotation Gates	9
2.2.3 The CNOT Gate	9
2.3 Observables, Expectation Values, and Measurement	9
2.4 The Variational Principle	10
2.5 The Transverse-Field Ising Model	10
2.5.1 Hamiltonian as a Sparse Pauli Operator	11
2.6 The Barren Plateau Phenomenon in Detail	11
2.6.1 Mathematical Statement	11
2.6.2 Physical Interpretation	11
2.6.3 Empirical Detection	11
3 Classical Machine Learning Foundations	13
3.1 Feedforward Neural Networks	13
3.1.1 Activation Functions	13
3.1.2 Weight Initialisation	14
3.2 The Parameter-Shift Rule	14
3.3 Backpropagation Through the Quantum Layer	14
3.4 The Adam Optimiser	15
4 Hardware-Efficient Ansatz Design	16
4.1 Motivation for Hardware-Efficient Circuits	16
4.2 Structure of the Ansatz	16
4.2.1 Single-Qubit Rotation Block	16
4.2.2 Entangling Block	16
4.2.3 Total Parameter Count	16
4.3 Statevector Simulation	17
5 Training Module: <code>Training.py</code>	18

5.1	Overview and Design Philosophy	18
5.2	Configuration Block	18
5.3	Hamiltonian Construction: <code>make_hamiltonian</code>	18
5.4	Energy Computation: <code>energy</code>	19
5.5	Gradient Computation: <code>gradient</code>	20
5.6	Adam Optimiser: <code>adam_step</code>	20
5.7	Neural Network: Forward and Backward Passes	21
5.7.1	<code>make_nn</code> — Weight Initialisation	21
5.7.2	<code>nn_forward</code> — Forward Pass	21
5.7.3	<code>nn_backward</code> — Backward Pass	21
5.8	Experiment 1: Gradient Variance Benchmark	22
5.9	Experiment 2: Energy Convergence and Model Saving	23
5.10	Plotting and Summary	24
6	Production Pipeline: <code>pipeline.py</code>	25
6.1	Architectural Overview	25
6.2	Global Configuration and Shared Helpers	25
6.2.1	<code>make_run_dir</code> — Timestamped Output Directory	25
6.2.2	<code>exact_ground_energy</code> — Diagonalisation	26
6.2.3	<code>energy</code> and <code>gradient</code> — Extended Signatures	26
6.3	Model Loading and Inference	26
6.3.1	<code>load_model</code>	26
6.3.2	<code>generate_params</code> — Batched Inference	27
6.4	Pipeline 1: VQE with NN Warm-Start	27
6.5	Pipeline 2: Hamiltonian Transfer Benchmark	28
6.5.1	Scientific Rationale	28
6.5.2	Heatmap Visualisation	29
6.5.3	Result Persistence	29
6.6	Pipeline 3: Parameter Generation and Ranking	29
6.6.1	Use Case	29
6.6.2	Output Formats	29
6.7	Pipeline 4: Model Export	30
6.7.1	JSON Export	30
6.7.2	CSV Export	30
6.7.3	PyTorch Export	30
6.7.4	Raw VQC Parameter Export	31
6.8	Command-Line Interface	31
7	Results and Analysis	32
7.1	Gradient Variance Benchmarking	32
7.1.1	Observed Variance Decay	33
7.1.2	Interpretation	34
7.2	Energy Convergence	34
7.2.1	Shallow Circuit: $L = 4$	34
7.2.2	Deep Circuit: $L = 10$	34
7.3	Hamiltonian Transfer Benchmark	35
7.3.1	Numerical Results	35
7.3.2	Discussion	36

7.4	Candidate Parameter Generation	36
7.4.1	Top-5 Ranked Candidates	37
7.4.2	Discussion	37
7.5	VQE Pipeline Convergence	38
7.6	Summary of Results	38
8	Software Engineering Considerations	40
8.1	Code Organisation	40
8.1.1	Module Separation	40
8.1.2	Pure NumPy Implementation	40
8.2	Reproducibility	40
8.3	Error Handling and Robustness	41
8.4	Numerical Considerations	41
8.4.1	Floating-Point Precision	41
8.4.2	Gradient Clipping	41
8.4.3	Energy Landscape Non-Convexity	41
9	Comparison with Related Approaches	42
9.1	Alternative Barren Plateau Mitigation Strategies	42
9.1.1	Layer-by-Layer Training	42
9.1.2	Correlated Parameter Initialisation	42
9.1.3	Quantum Natural Gradient	42
9.1.4	NN-Based Initialisation (This Work)	42
9.2	Comparison to Classical Optimisation Heuristics	43
10	Conclusion and Future Work	44
10.1	Summary of Contributions	44
10.2	Future Directions	44
10.2.1	Scaling Beyond 6 Qubits	44
10.2.2	Deeper Neural Networks	44
10.2.3	Generative Models	45
10.2.4	Hardware Noise Modelling	45
10.2.5	Alternative Ansätze	45
10.2.6	Automated Hyperparameter Tuning	45
A	Full Annotated Code: <code>Training.py</code>	46
B	Full Annotated Code: <code>pipeline.py</code>	49
C	Mathematical Reference	50
C.1	Pauli Algebra	50
C.2	Matrix Exponential for Rotations	50
C.3	Derivation of the Parameter-Shift Rule	50
C.4	Adam Convergence Properties	50
	Bibliography	51

List of Abbreviations

Abbreviation	Full Form
VQC	Variational Quantum Circuit
VQE	Variational Quantum Eigensolver
VQA	Variational Quantum Algorithm
NISQ	Noisy Intermediate-Scale Quantum
NN	Neural Network
DNN	Deep Neural Network
TFIM	Transverse-Field Ising Model
BP	Barren Plateau
Adam	Adaptive Moment Estimation
QPU	Quantum Processing Unit
PSR	Parameter-Shift Rule
SPSA	Simultaneous Perturbation Stochastic Approximation
COBYLA	Constrained Optimisation by Linear Approximations
CLI	Command-Line Interface
JSON	JavaScript Object Notation
CSV	Comma-Separated Values
NPZ	NumPy compressed archive format
ReLU	Rectified Linear Unit
SGD	Stochastic Gradient Descent

Chapter 1

Introduction and Motivation

1.1 The Quantum Computing Landscape

The field of quantum computing promises to revolutionise computational science by exploiting phenomena that have no classical analogue. Unlike a classical bit, which occupies exactly one of two states $\{0, 1\}$, a quantum bit—a *qubit*—can exist in a continuous superposition of both states simultaneously. When many qubits are entangled, the combined Hilbert space grows as 2^n , enabling quantum algorithms to explore an exponentially large space of solutions in parallel.

Despite this promise, building large-scale, fault-tolerant quantum computers remains beyond current engineering capabilities. The devices available today belong to the *Noisy Intermediate-Scale Quantum* (NISQ) era: they possess on the order of tens to a few hundred qubits, and every gate operation introduces some error due to decoherence and imperfect control electronics. These two limitations—qubit count and gate fidelity—place a strict upper bound on the depth of quantum circuits that can be executed with meaningful signal.

1.2 Variational Quantum Algorithms

To extract useful computation from NISQ devices, the community has developed the class of *Variational Quantum Algorithms* (VQAs). VQAs follow a classical feedback loop:

1. A parameterised quantum circuit (the *ansatz*) is executed on the quantum device, producing a quantum state $|\psi(\boldsymbol{\theta})\rangle$ for a given vector of real-valued parameters $\boldsymbol{\theta} \in \mathbb{R}^M$.
2. The quantum device is measured to evaluate a cost function $C(\boldsymbol{\theta}) = \langle \psi(\boldsymbol{\theta}) | \hat{H} | \psi(\boldsymbol{\theta}) \rangle$.
3. A classical optimiser (e.g., gradient descent) updates $\boldsymbol{\theta}$ to minimise C .
4. Steps 1–3 repeat until convergence.

The most celebrated VQA is the *Variational Quantum Eigensolver* (VQE), which targets the ground state energy E_0 of a Hamiltonian $\hat{H}$. VQE is foundational in quantum chemistry and condensed matter physics.

1.3 The Barren Plateau Crisis

In 2018, McClean et al. identified a catastrophic training obstacle in deep VQAs that they named the *barren plateau*. The phenomenon is mathematically sharp: for a randomly

initialised n -qubit circuit of depth L , the variance of the gradient of the cost function decays exponentially in both n and L :

$$\text{Var}\left[\frac{\partial C}{\partial \theta_k}\right] \in \mathcal{O}\left(\frac{1}{b^n}\right), \quad b > 1. \quad (1.1)$$

In practice, once the circuit exceeds ~ 10 qubits or a few dozen layers, the gradient signal is so tiny that an exponential number of circuit evaluations (shots) would be needed to resolve it above statistical noise. The classical optimiser effectively wanders on a featureless flat landscape, unable to find any downhill direction.

1.4 The Neural Network Mitigation Strategy

Friedrich and Maziero (2022) proposed an elegant solution: instead of sampling θ uniformly at random, train a classical neural network to generate θ values that are biased toward low-energy, high-gradient regions of the cost landscape. The key insight is that the neural network acts as a *structured prior*: it restricts the initialisation to a sub-manifold of parameter space that the network has learned to associate with non-flat cost landscapes.

1.5 Project Scope and Report Structure

This report documents the complete implementation of this strategy across two Python modules:

- **Training.py** — the core training and benchmarking module: builds the TFIM Hamiltonian, runs the VQE ansatz, trains the NN via quantum backpropagation, benchmarks gradient variance, and compares energy convergence.
- **pipeline.py** — the production inference pipeline: loads the saved NN models and exposes four distinct modes of operation for downstream scientific use.

The remainder of this report is organised as follows. Chapter 2 lays out all required quantum mechanics from first principles. Chapter 3 introduces the machine learning components. Chapter 5 dissects **Training.py** in full. Chapter 6 covers every pipeline in **pipeline.py**. Chapter 7 analyses the experimental results. Chapter 10 concludes with future directions.

Chapter 2

Quantum Mechanics Foundations

2.1 State Vectors and the Hilbert Space

Quantum mechanics describes a physical system's state as a vector, called a *ket*, in a complex Hilbert space $\mathcal{H}$. For an n -qubit system, the Hilbert space has dimension 2^n :

$$\mathcal{H} = \underbrace{\mathbb{C}^2 \otimes \mathbb{C}^2 \otimes \dots \otimes \mathbb{C}^2}_{n \text{ times}} \cong \mathbb{C}^{2^n}. \quad (2.1)$$

A single qubit lives in $\mathbb{C}^2$, spanned by the computational basis states:

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}. \quad (2.2)$$

The most general single-qubit state is a *superposition*:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \quad \alpha, \beta \in \mathbb{C}, \quad |\alpha|^2 + |\beta|^2 = 1. \quad (2.3)$$

$|\alpha|^2$ and $|\beta|^2$ are the probabilities of measuring the qubit in state $|0\rangle$ or $|1\rangle$, respectively.

2.1.1 Multi-Qubit States and Tensor Products

For an n -qubit system, the computational basis is formed by all 2^n binary strings $|b_{n-1} \dots b_1 b_0\rangle$, where each $b_i \in \{0, 1\}$. A general state is:

$$|\Psi\rangle = \sum_{x=0}^{2^n-1} c_x |x\rangle, \quad \sum_x |c_x|^2 = 1. \quad (2.4)$$

The tensor product of two single-qubit states $|\psi_A\rangle \otimes |\psi_B\rangle$ has components:

$$\begin{pmatrix} \alpha_A \\ \beta_A \end{pmatrix} \otimes \begin{pmatrix} \alpha_B \\ \beta_B \end{pmatrix} = \begin{pmatrix} \alpha_A \alpha_B \\ \alpha_A \beta_B \\ \beta_A \alpha_B \\ \beta_A \beta_B \end{pmatrix}. \quad (2.5)$$

States that *cannot* be written as such a product for any bipartition are called *entangled*. Entanglement is the resource exploited by quantum algorithms to achieve correlations beyond classical reach.

2.2 Quantum Gates and Unitary Evolution

State evolution in a closed quantum system is described by unitary operators U , satisfying $UU^\dagger = U^\dagger U = \mathbb{I}$. Single-qubit gates of particular importance in this project are:

2.2.1 Pauli Matrices

The three Pauli matrices and the identity form a complete basis for 2×2 Hermitian operators:

$$\mathbb{I} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}. \quad (2.6)$$

X is the quantum analogue of a NOT gate, flipping $|0\rangle \leftrightarrow |1\rangle$. Z is the phase-flip gate: $Z|0\rangle = |0\rangle$, $Z|1\rangle = -|1\rangle$. These Pauli operators also generate rotation gates via the matrix exponential.

2.2.2 Rotation Gates

Rotation by angle θ about the $\hat{y}$ -axis of the Bloch sphere:

$$R_y(\theta) = e^{-i\theta Y/2} = \begin{pmatrix} \cos(\theta/2) & -\sin(\theta/2) \\ \sin(\theta/2) & \cos(\theta/2) \end{pmatrix}. \quad (2.7)$$

Rotation by angle θ about the $\hat{z}$ -axis:

$$R_z(\theta) = e^{-i\theta Z/2} = \begin{pmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{pmatrix}. \quad (2.8)$$

An arbitrary single-qubit rotation can be decomposed as $R_z(\alpha)R_y(\beta)R_z(\gamma)$ for appropriate angles. The combination $R_y(\theta_1)R_z(\theta_2)$ used in our ansatz provides two free parameters per qubit per layer, which is exactly the approach taken in `Training.py` and `pipeline.py`.

2.2.3 The CNOT Gate

The Controlled-NOT (CNOT) gate acts on two qubits—a *control* qubit c and a *target* qubit t —and flips the target if and only if the control is $|1\rangle$:

$$\text{CNOT}|c, t\rangle = |c, c \oplus t\rangle, \quad (2.9)$$

where $\oplus$ denotes addition modulo 2. In matrix form (in the basis $\{|00\rangle, |01\rangle, |10\rangle, |11\rangle\}$):

$$\text{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}. \quad (2.10)$$

The CNOT gate is the standard entangling gate used in hardware-efficient ansätze.

2.3 Observables, Expectation Values, and Measurement

An observable is a Hermitian operator $\hat{O} = \hat{O}^\dagger$. By the spectral theorem, it has real eigenvalues λ_k and orthonormal eigenstates $|k\rangle$, so $\hat{O} = \sum_k \lambda_k |k\rangle\langle k|$. The *expectation value* of $\hat{O}$ in state $|\psi\rangle$ is:

$$\langle \hat{O} \rangle_\psi = \langle \psi | \hat{O} | \psi \rangle = \sum_k \lambda_k |\langle k | \psi \rangle|^2. \quad (2.11)$$

This is the quantum mechanical analogue of a weighted average, where $|\langle k|\psi\rangle|^2$ is the probability of measuring eigenvalue λ_k . In the code, the energy expectation value is computed exactly (without statistical shot noise) as:

$$\langle H \rangle = \mathbf{sv}^\dagger \cdot H_{\text{mat}} \cdot \mathbf{sv}, \quad (2.12)$$

where $\mathbf{sv}$ is the complex statevector of length 2^n obtained from Qiskit's `Statevector` simulator, and H_{mat} is the $2^n \times 2^n$ matrix representation of the Hamiltonian.

2.4 The Variational Principle

The cornerstone of VQE is the *Rayleigh-Ritz variational principle*:

Theorem 2.1 (Variational Principle). *For any normalised trial state $|\psi\rangle$, the expectation value of a Hamiltonian $\hat{H}$ satisfies:*

$$\langle \psi | \hat{H} | \psi \rangle \geq E_0, \quad (2.13)$$

where E_0 is the ground state energy (lowest eigenvalue) of $\hat{H}$.

Proof. Expand $|\psi\rangle$ in the eigenbasis $\{|E_k\rangle\}$ of $\hat{H}$: $|\psi\rangle = \sum_k c_k |E_k\rangle$. Then:

$$\langle \psi | \hat{H} | \psi \rangle = \sum_k |c_k|^2 E_k \geq E_0 \sum_k |c_k|^2 = E_0. \quad (2.14)$$

□

Equality holds iff $|\psi\rangle = |E_0\rangle$ (the true ground state). By parameterising $|\psi(\boldsymbol{\theta})\rangle$ via a VQC and minimising $C(\boldsymbol{\theta}) = \langle \psi(\boldsymbol{\theta}) | \hat{H} | \psi(\boldsymbol{\theta}) \rangle$, we systematically approach E_0 from above.

2.5 The Transverse-Field Ising Model

The 1D Transverse-Field Ising Model (TFIM) is a paradigmatic model in condensed matter physics that exhibits a quantum phase transition as the ratio h/J crosses a critical value. For n qubits with periodic boundary conditions, the Hamiltonian is:

$$\hat{H} = -J \sum_{i=0}^{n-1} Z_i Z_{(i+1) \bmod n} - h \sum_{i=0}^{n-1} X_i. \quad (2.15)$$

where:

- $Z_i = \mathbb{I}^{\otimes i} \otimes Z \otimes \mathbb{I}^{\otimes (n-i-1)}$ acts as a Pauli- Z on qubit i and as identity on all others.
- X_i similarly acts as Pauli- X on qubit i .
- $J \geq 0$ is the ferromagnetic coupling strength.
- $h \geq 0$ is the transverse magnetic field strength.

The first sum penalises configurations where adjacent qubits anti-align in the z -basis (the Ising interaction). The second sum drives the system toward the x -basis, competing with the first. At $h = 0$, the ground state is a doubly degenerate ferromagnet ($|00 \dots 0\rangle$ or $|11 \dots 1\rangle$). At $h/J \rightarrow \infty$, the ground state approaches the product state $|+\rangle^{\otimes n}$ where $|+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$.

2.5.1 Hamiltonian as a Sparse Pauli Operator

For a 6-qubit system, $\hat{H}$ lives in a 64×64 -dimensional space. Qiskit represents it efficiently as a `SparsePauliOp`: a weighted sum of Pauli strings. Each Pauli string is a tensor product of single-site Paulis written in Qiskit’s reversed qubit ordering. For example, the ZZ term on qubits 0 and 1 in a 6-qubit system (Qiskit ordering: qubit 0 is rightmost) is the string `IIII ZZ`.

The exact ground state energy is obtained by diagonalising H_{mat} numerically:

$$E_0 = \min \text{eigvals}(H_{\text{mat}}) = \text{np.linalg.eigvalsh}(H_{\text{mat}})[0]. \quad (2.16)$$

For $n = 6$, $J = 1.0$, $h = 0.5$, this gives $E_0 = -6.385$ (to 3 d.p.).

2.6 The Barren Plateau Phenomenon in Detail

2.6.1 Mathematical Statement

Consider a cost function of the form $C(\boldsymbol{\theta}) = \langle \psi(\boldsymbol{\theta}) | \hat{O} | \psi(\boldsymbol{\theta}) \rangle$ where $|\psi(\boldsymbol{\theta})\rangle$ is prepared by a random quantum circuit of depth L on n qubits. McClean et al. proved that if the circuit forms a 2-design (i.e., it is sufficiently expressive), then:

$$\mathbb{E}_{\boldsymbol{\theta}} \left[\frac{\partial C}{\partial \theta_k} \right] = 0, \quad (2.17)$$

and more critically:

$$\text{Var}_{\boldsymbol{\theta}} \left[\frac{\partial C}{\partial \theta_k} \right] \leq \frac{2 \|\hat{O}\|_F^2}{2^n}, \quad (2.18)$$

where $\|\hat{O}\|_F$ is the Frobenius norm of $\hat{O}$. Equation (2.18) shows that the variance of any gradient component decays *exponentially* in the number of qubits n . For $n = 6$, the denominator $2^6 = 64$; for $n = 20$, it exceeds one million.

2.6.2 Physical Interpretation

The barren plateau can be understood geometrically. In a landscape of M parameters, the typical gradient magnitude is $\mathcal{O}(2^{-n/2})$. A gradient this small means that, relative to the scale of the cost function, the loss surface looks utterly flat in all directions. A classical optimiser navigating this surface cannot tell which direction leads downhill without an exponentially precise measurement, which demands an exponentially large number of circuit shots. Since NISQ devices have finite coherence times, this is physically infeasible.

2.6.3 Empirical Detection

The code detects BPs empirically by sampling $N_{\text{samples}} = 150$ independent random parameter vectors $\boldsymbol{\theta}^{(i)}$, computing the first component of the gradient $\partial C / \partial \theta_0^{(i)}$ via the parameter-shift rule, and then computing the sample variance:

$$\widehat{\text{Var}} \left[\frac{\partial C}{\partial \theta_0} \right] = \frac{1}{N_{\text{samples}} - 1} \sum_{i=1}^{N_{\text{samples}}} (g_i - \bar{g})^2, \quad (2.19)$$

where $g_i = \partial C / \partial \theta_0 \big|_{\theta = \theta^{(i)}}$ and $\bar{g}$ is the sample mean. This is repeated for circuit depths $L = 1, 2, \dots, 12$ to produce the variance-vs-depth curve.

Chapter 3

Classical Machine Learning Foundations

3.1 Feedforward Neural Networks

A feedforward neural network is a composition of affine transformations and pointwise nonlinearities. Given an input vector $\mathbf{z} \in \mathbb{R}^{d_{\text{in}}}$, the 2-layer network used in this project computes:

$$\mathbf{h} = \sigma_1(\mathbf{z}\mathbf{W}_1 + \mathbf{b}_1), \quad (3.1)$$

$$\boldsymbol{\theta} = \sigma_2(\mathbf{h}\mathbf{W}_2 + \mathbf{b}_2), \quad (3.2)$$

where:

- $\mathbf{W}_1 \in \mathbb{R}^{d_{\text{in}} \times d_{\text{hid}}}$, $\mathbf{b}_1 \in \mathbb{R}^{d_{\text{hid}}}$ are the first layer's weight matrix and bias vector.
- $\mathbf{W}_2 \in \mathbb{R}^{d_{\text{hid}} \times d_{\text{out}}}$, $\mathbf{b}_2 \in \mathbb{R}^{d_{\text{out}}}$ are the second layer's weight matrix and bias vector.
- $\sigma_1 = \text{ReLU}$ (Rectified Linear Unit).
- $\sigma_2 = \pi \tanh(\cdot)$ (hyperbolic tangent scaled by π).

3.1.1 Activation Functions

ReLU. The Rectified Linear Unit is defined as:

$$\text{ReLU}(x) = \max(0, x) = \begin{cases} x & x > 0, \\ 0 & x \leq 0. \end{cases} \quad (3.3)$$

ReLU is applied elementwise to $\mathbf{z}\mathbf{W}_1 + \mathbf{b}_1$. It is computationally cheap, empirically effective at alleviating the vanishing gradient problem, and easy to differentiate (its sub-gradient is the Heaviside step function). Its derivative is:

$$\frac{d}{dx} \text{ReLU}(x) = \mathbb{I}[x > 0]. \quad (3.4)$$

$\pi \tanh$. The hyperbolic tangent outputs values in $(-1, 1)$:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}. \quad (3.5)$$

Multiplying by π maps the output to $(-\pi, \pi)$, which is exactly the natural domain of quantum rotation gate angles. This choice of output activation is physically motivated:

rotation gates $R_y(\theta)$ and $R_z(\theta)$ are 2π -periodic, so constraining $\theta \in (-\pi, \pi)$ avoids redundancy. The derivative of $\pi \tanh$ is:

$$\frac{d}{dx}[\pi \tanh(x)] = \pi (1 - \tanh^2(x)) = \frac{\pi}{\cosh^2(x)}. \quad (3.6)$$

3.1.2 Weight Initialisation

The He initialisation strategy is used for weights:

$$W_{ij}^{(1)} \sim \mathcal{N}\left(0, \frac{2}{d_{\text{in}}}\right), \quad W_{ij}^{(2)} \sim \mathcal{N}\left(0, \frac{2}{d_{\text{hid}}}\right). \quad (3.7)$$

This is the standard initialisation for networks with ReLU activations: scaling by $\sqrt{2/d}$ ensures that the variance of activations does not explode or vanish as signals pass through the network, facilitating stable initial gradient flow.

3.2 The Parameter-Shift Rule

To train the neural network using the quantum circuit's gradient, we need an analytical formula for $\partial C / \partial \theta_k$. The *parameter-shift rule* (PSR) provides this:

Key Concept

For a cost function $C(\boldsymbol{\theta}) = \langle \psi(\boldsymbol{\theta}) | \hat{H} | \psi(\boldsymbol{\theta}) \rangle$ where θ_k appears in the generator of a unitary via $e^{-i\theta_k G}$ with G having eigenvalues $\pm \frac{1}{2}$, the exact gradient is:

$$\frac{\partial C}{\partial \theta_k} = \frac{1}{2} \left[C\left(\boldsymbol{\theta} + \frac{\pi}{2} \hat{\mathbf{e}}_k\right) - C\left(\boldsymbol{\theta} - \frac{\pi}{2} \hat{\mathbf{e}}_k\right) \right]. \quad (3.8)$$

This formula is exact (not an approximation like finite differences) and is hardware-compatible, requiring only two circuit evaluations per parameter. The rotation gates R_y and R_z satisfy this condition because their generators ($Y/2$ and $Z/2$) have eigenvalues $\pm 1/2$.

In the code, the gradient of all M parameters is computed as:

$$\mathbf{g} = \left(\frac{\partial C}{\partial \theta_1}, \frac{\partial C}{\partial \theta_2}, \dots, \frac{\partial C}{\partial \theta_M} \right), \quad (3.9)$$

requiring $2M$ circuit evaluations total (two per parameter).

3.3 Backpropagation Through the Quantum Layer

The neural network is trained end-to-end: the loss is the quantum energy $C(\boldsymbol{\theta}) = \langle H \rangle$, and gradients flow backwards through the NN computation graph. The chain rule gives:

$$\frac{\partial C}{\partial W_{jk}^{(2)}} = \frac{\partial C}{\partial \theta_k} \cdot \frac{\partial \theta_k}{\partial (h \mathbf{W}_2 + \mathbf{b}_2)_k} \cdot h_j = g_k^q \cdot \pi(1 - \tanh^2(\text{pre2}_k)) \cdot h_j, \quad (3.10)$$

where $g_k^q = \partial C / \partial \theta_k$ is the quantum gradient (from PSR) and $\text{pre2}_k = (\mathbf{h}\mathbf{W}_2 + \mathbf{b}_2)_k$ is the pre-activation of the output layer.

In vectorised form (using outer products):

$$\mathbf{d}_2 = \mathbf{g}^q \odot \pi(1 - \tanh^2(\text{pre2})) \quad (\text{output delta}) \quad (3.11)$$

$$\Delta \mathbf{W}_2 = \mathbf{h}^\top \otimes \mathbf{d}_2 \quad (\text{weight gradient}) \quad (3.12)$$

$$\mathbf{d}_1 = (\mathbf{d}_2 \mathbf{W}_2^\top) \odot \mathbb{K}[\text{pre1} > 0] \quad (\text{hidden delta}) \quad (3.13)$$

$$\Delta \mathbf{W}_1 = \mathbf{z}^\top \otimes \mathbf{d}_1 \quad (\text{weight gradient}) \quad (3.14)$$

Weights are updated by gradient descent: $\mathbf{W} \leftarrow \mathbf{W} - \eta \Delta \mathbf{W}$, with learning rate $\eta = 2 \times 10^{-3}$.

3.4 The Adam Optimiser

Adam (Adaptive Moment Estimation) is the classical optimiser used for VQE fine-tuning. It maintains exponentially weighted moving averages of both the gradient m_t and the squared gradient v_t :

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad (3.15)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \quad (3.16)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}, \quad (3.17)$$

$$\boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}. \quad (3.18)$$

Table 3.1: Adam hyperparameter values used in both modules.

Symbol	Meaning	Value
β_1	First-moment decay	0.9
β_2	Second-moment decay	0.999
ϵ	Numerical stability	10^{-8}
α	Learning rate (<code>Training.py</code>)	0.03–0.05
α	Learning rate (<code>pipeline.py</code>)	0.03

The bias-correction terms in (3.17) are essential during the early iterations, when m_t and v_t are still close to zero (initialised as such) and would otherwise underestimate the true moment magnitudes.

Chapter 4

Hardware-Efficient Ansatz Design

4.1 Motivation for Hardware-Efficient Circuits

The choice of ansatz is critical in VQE. A circuit must be:

1. Expressive enough to represent the target ground state.
2. Shallow enough to execute on NISQ hardware without prohibitive gate errors.
3. Compatible with the physical qubit connectivity of the target device.

Hardware-efficient ansätze (HEA) are designed to satisfy conditions 2 and 3 by using only the native gate set of the target device and connecting only physically adjacent qubits. While they may sacrifice some expressibility, they are the practical choice for near-term quantum hardware.

4.2 Structure of the Ansatz

For n qubits and L layers, the ansatz is built as follows.

4.2.1 Single-Qubit Rotation Block

In each layer $\ell = 1, \dots, L$, every qubit $q = 0, \dots, n - 1$ undergoes two rotation gates:

$$U_q^{(\ell)}(\theta_{2(q+n\ell)}, \theta_{2(q+n\ell)+1}) = R_z(\theta_{2(q+n\ell)+1}) \cdot R_y(\theta_{2(q+n\ell)}). \quad (4.1)$$

This contributes $2n$ parameters per layer.

4.2.2 Entangling Block

After the rotation block, a linear chain of CNOT gates creates entanglement:

$$\text{CNOT}_{0 \rightarrow 1} \cdot \text{CNOT}_{1 \rightarrow 2} \cdots \text{CNOT}_{(n-2) \rightarrow (n-1)}. \quad (4.2)$$

This connects each qubit to its right neighbour, forming a minimal entangling structure that avoids the need for long-range interactions.

4.2.3 Total Parameter Count

The total number of trainable parameters for n qubits and L layers is:

$$M = 2 \times n \times L. \quad (4.3)$$

For $n = 6$:

Table 4.1: Parameter count as a function of circuit depth.

L	1	2	4	6	8	10	12
M	12	24	48	72	96	120	144

4.3 Statevector Simulation

In the code, the circuit is not run on real hardware but simulated exactly using Qiskit’s **Statevector** class. This class evolves the full 2^n -dimensional state vector through each gate via matrix-vector multiplication:

$$|\psi_{\text{final}}\rangle = U_L U_{L-1} \cdots U_1 |\mathbf{0}\rangle, \quad (4.4)$$

where $|\mathbf{0}\rangle = |00 \cdots 0\rangle$ is the all-zero initial state and each U_ℓ is the unitary corresponding to layer ℓ . The energy is then computed via (2.12) using NumPy matrix operations.

Chapter 5

Training Module: Training.py

5.1 Overview and Design Philosophy

Training.py is a self-contained research script. It runs two sequential experiments:

1. **Gradient Variance Benchmark:** Quantifies the barren plateau effect and demonstrates NN mitigation across circuit depths $L = 1$ to $L = 12$.
2. **Energy Convergence Comparison:** Compares VQE optimisation trajectories for random vs. NN-generated initialisation at two representative depths ($L = 4$ and $L = 10$).

At the end of Experiment 2, the trained NN models are saved as .npz files for downstream use by pipeline.py.

5.2 Configuration Block

```
1 N_QUBITS      = 6          # Forces the barren plateau to appear
2 J             = 1.0        # ZZ coupling strength
3 H_FIELD       = 0.5        # transverse field strength
4 MAX_DEPTH     = 12         # Extended to show the gradient decay
5                 clearly
6 N_SAMPLES     = 150        # Smooths out the zigzagging variance lines
7 NN_TRAIN      = 80         # Gives the neural network more time to
8                 learn
9 CONV_ITERS    = 200        # Gives the final fine-tuning enough runway
10 CONV_DEPTHS  = [4, 10]    # Compare a safe depth (4) vs a plateau
11                 depth (10)
12 SEED         = 42
```

Listing 5.1: Global configuration in Training.py

Every hyperparameter is placed at the top of the file for easy modification. The choice $n = 6$ is deliberate: it is large enough for barren plateaus to manifest clearly (recall the $1/2^n$ scaling), yet small enough for statevector simulation on a laptop CPU without GPU acceleration.

5.3 Hamiltonian Construction: make_hamiltonian

```
1 def make_hamiltonian(n, J=1.0, h=0.5):
2     terms = []
```

```

3     for i in range(n):
4         j = (i+1) % n                                # periodic boundary
5         p = ['I']*n; p[i]='Z'; p[j]='Z'
6         terms.append((''.join(reversed(p)), -J))
7     for i in range(n):
8         p = ['I']*n; p[i]='X'
9         terms.append((''.join(reversed(p)), -h))
10    return SparsePauliOp.from_list(terms)

```

Listing 5.2: `make_hamiltonian` — constructing the TFIM Hamiltonian as a `SparsePauliOp`.

Step-by-step walkthrough.

- **Lines 3–6:** For each qubit i , compute its right neighbour $j = (i + 1) \bmod n$ (modular arithmetic enforces periodic boundary conditions, so qubit $n - 1$ couples to qubit 0). Build the Pauli string for $Z_i Z_j$: start from all identities and set position i and j to 'Z'. The coefficient is $-J$.
- **Qiskit qubit ordering:** Qiskit labels the rightmost character of a Pauli string as qubit 0. Therefore, `reversed(p)` is necessary to align the physical qubit index with Qiskit's convention.
- **Lines 7–9:** Similarly build the X_i terms with coefficient $-h$.
- **Line 10:** Combine all terms into a `SparsePauliOp` object, which Qiskit can convert to a dense matrix via `.to_matrix()`.

The negative signs in the Hamiltonian (2.15) mean the ZZ and X terms together drive the energy down, so the ground state has $\langle H \rangle < 0$.

5.4 Energy Computation: energy

```

1 def energy(params, n_layers):
2     qc = QuantumCircuit(N_QUBITS)
3     idx = 0
4     for _ in range(n_layers):
5         for q in range(N_QUBITS):
6             qc.ry(params[idx], q); idx += 1
7             qc.rz(params[idx], q); idx += 1
8         for q in range(N_QUBITS - 1):
9             qc.cx(q, q+1)
10    sv = Statevector(qc).data
11    return float(np.real(np.conj(sv) @ Hmat @ sv))

```

Listing 5.3: `energy` — evaluate the expectation value $\langle H \rangle$.

Analysis.

- A fresh `QuantumCircuit` is created at every function call, starting from the all-zero state $|000000\rangle$.

- The parameter index `idx` increments sequentially, mapping flat parameter array entries to specific qubits and layers in column-major order: $(q, \ell) \mapsto 2(q + n\ell)$ for R_y and $2(q + n\ell) + 1$ for R_z .
- CNOT gates between neighbouring qubits (but not qubit $n - 1 \rightarrow 0$, which would require a longer-range gate) create entanglement.
- `Statevector(qc).data` returns the full complex statevector as a NumPy array.
- The expectation value is computed by the matrix sandwich formula: $\langle H \rangle = \text{sv}^\dagger \cdot H_{\text{mat}} \cdot \text{sv}$.
- `np.real()` discards the numerically-zero imaginary part (Hamiltonian is Hermitian, so the expectation value is real).

Computational note: This function is called $2M$ times per gradient evaluation (M parameters) and N_{samples} times per variance sample. With $M = 120$ (at $L = 10$) and $N_{\text{samples}} = 150$, the total call count in Experiment 1 alone is on the order of $\sum_{L=1}^{12} 2 \times 2nL \times 150 \approx 235,800$ circuit simulations—a significant but tractable load for a statevector simulator.

5.5 Gradient Computation: `gradient`

```

1 def gradient(params, n_layers):
2     g = np.zeros_like(params)
3     for i in range(len(params)):
4         pp = params.copy(); pp[i] += np.pi/2
5         pm = params.copy(); pm[i] -= np.pi/2
6         g[i] = (energy(pp, n_layers) - energy(pm, n_layers)) / 2
7     return g

```

Listing 5.4: `gradient` — parameter-shift rule gradient.

This is a direct implementation of (3.8). Note that `params.copy()` is essential: without copying, the in-place additions would corrupt the original parameter vector. Each call to `gradient` makes $2M$ calls to `energy`.

5.6 Adam Optimiser: `adam_step`

```

1 def adam_step(params, g, m, v, t, lr=0.05):
2     b1, b2, eps = 0.9, 0.999, 1e-8
3     m = b1*m + (1-b1)*g
4     v = b2*v + (1-b2)*g**2
5     t += 1
6     params = params - lr*(m/(1-b1**t))/(np.sqrt(v/(1-b2**t))+eps)
7     return params, m, v, t

```

Listing 5.5: `adam_step` — one step of Adam.

This implements equations (3.15)–(3.18). The function returns updated copies of both `params` and the moment estimates m, v , which the caller must store and pass in the next

step. The step counter t is also incremented and returned, since the bias correction factors $(1 - \beta_1^t)$ and $(1 - \beta_2^t)$ depend on it.

5.7 Neural Network: Forward and Backward Passes

5.7.1 make_nn — Weight Initialisation

```

1 def make_nn(input_dim, output_dim, hidden=32):
2     W1 = np.random.randn(input_dim, hidden) * np.sqrt(2/input_dim)
3     b1 = np.zeros(hidden)
4     W2 = np.random.randn(hidden, output_dim) * np.sqrt(2/hidden)
5     b2 = np.zeros(output_dim)
6     return W1, b1, W2, b2

```

Listing 5.6: make_nn — He initialisation of NN weights.

The network architecture is:

$$\underbrace{8}_{\text{input}} \xrightarrow{W_1, b_1} \underbrace{32}_{\text{hidden}} \xrightarrow{\text{ReLU}} \underbrace{32}_{\text{hidden}} \xrightarrow{W_2, b_2} \underbrace{M}_{\text{output}} \xrightarrow{\pi \tanh} \underbrace{M}_{\theta}. \quad (5.1)$$

The input dimension 8 is a random noise vector $\mathbf{z} \sim \mathcal{N}(0, \mathbb{I}_8)$, playing the role of a latent code. The output dimension $M = 2nL$ matches the number of VQC parameters.

5.7.2 nn_forward — Forward Pass

```

1 def nn_forward(z, W1, b1, W2, b2):
2     h = np.maximum(0, z @ W1 + b1)           # ReLU hidden
3     return np.tanh(h @ W2 + b2) * np.pi     # Tanh*pi output

```

Listing 5.7: nn_forward — compute θ from noise $\mathbf{z}$.

$\mathbf{z} @ \mathbf{W}_1$ computes the matrix product $\mathbf{z}\mathbf{W}_1$ (row vector times matrix), yielding a length-32 hidden pre-activation. $\text{np.maximum}(0, \cdot)$ applies ReLU elementwise. The output is then bounded to $(-\pi, \pi)$ by $\pi \tanh$.

5.7.3 nn_backward — Backward Pass

```

1 def nn_backward(z, W1, b1, W2, b2, gq, lr=2e-3):
2     h = np.maximum(0, z @ W1 + b1)
3     pre2 = h @ W2 + b2
4     d2 = gq * (1 - np.tanh(pre2)**2) * np.pi
5     dW2 = np.outer(h, d2); db2 = d2
6     d1 = (d2 @ W2.T) * (h > 0)
7     dW1 = np.outer(z, d1); db1 = d1
8     W1 -= lr * dW1; b1 -= lr * db1
9     W2 -= lr * dW2; b2 -= lr * db2
10    return W1, b1, W2, b2

```

Listing 5.8: nn_backward — gradient descent update of NN weights.

Detailed derivation of d2. Let $\text{pre2}_k = (\mathbf{h}\mathbf{W}_2 + \mathbf{b}_2)_k$ and $\theta_k = \pi \tanh(\text{pre2}_k)$. By the chain rule:

$$\frac{\partial C}{\partial \text{pre2}_k} = \underbrace{\frac{\partial C}{\partial \theta_k}}_{g_k^q} \cdot \frac{d(\pi \tanh)}{d(\text{pre2}_k)} = g_k^q \cdot \pi(1 - \tanh^2(\text{pre2}_k)). \quad (5.2)$$

This is exactly d2 in the code.

Weight gradients. The gradient of the loss w.r.t. $W_{jk}^{(2)}$ is:

$$\frac{\partial C}{\partial W_{jk}^{(2)}} = h_j \cdot d_{2,k} \implies \Delta \mathbf{W}_2 = \mathbf{h} \otimes \mathbf{d}_2 \quad (\text{outer product}). \quad (5.3)$$

Hidden delta. Backpropagating through the ReLU:

$$\frac{\partial C}{\partial \text{pre1}_j} = \sum_k d_{2,k} W_{jk}^{(2)} \cdot \mathbb{I}[h_j > 0] = (\mathbf{d}_2 \mathbf{W}_2^\top)_j \cdot \mathbb{I}[h_j > 0]. \quad (5.4)$$

This is d1, and similarly $\text{dW1} = \text{np.outer}(\mathbf{z}, \mathbf{d1})$.

5.8 Experiment 1: Gradient Variance Benchmark

```

1 for L in depths:
2     npt = n_params(L)
3
4     # Random initialisation
5     gs_r = []
6     for _ in range(N_SAMPLES):
7         t = np.random.uniform(-np.pi, np.pi, npt)
8         gs_r.append(gradient(t, L)[0])
9     var_rand.append(np.var(gs_r))
10
11    # NN-generated initialisation
12    W1, b1, W2, b2 = make_nn(8, npt, hidden=32)
13    for _ in range(N_TRAIN):
14        z = np.random.randn(8)
15        th = nn_forward(z, W1, b1, W2, b2)
16        gq = gradient(th, L)
17        W1, b1, W2, b2 = nn_backward(z, W1, b1, W2, b2, gq)
18
19    gs_n = []
20    for _ in range(N_SAMPLES):
21        z = np.random.randn(8)
22        th = nn_forward(z, W1, b1, W2, b2)
23        gs_n.append(gradient(th, L)[0])
24    var_nn.append(np.var(gs_n))

```

Listing 5.9: Core loop of Experiment 1.

Algorithmic flow. For each depth L :

1. **Random baseline:** Draw 150 random parameter vectors from $\mathcal{U}(-\pi, \pi)^M$, compute the first gradient component $\partial C / \partial \theta_0$ for each, and record the sample variance.
2. **NN pre-training:** Initialise a fresh NN with He-scaled random weights. For 80 iterations: sample noise $\mathbf{z}$, run forward pass to get parameters $\boldsymbol{\theta}$, compute the full gradient $\mathbf{g}^q$ via PSR, run backward pass to update NN weights. This trains the NN to output parameters that have large gradients.
3. **NN evaluation:** Sample 150 noise vectors, pass through the trained NN, compute first gradient component for each. Record the sample variance.

The key empirical question is: does the NN-generated variance remain significantly higher than the random-initialisation variance as L increases? If yes, the NN has successfully learned to avoid barren plateaus.

5.9 Experiment 2: Energy Convergence and Model Saving

```

1 for L in CONV_DEPTHS:
2     npt = n_params(L)
3
4     # NN pre-training
5     W1, b1, W2, b2 = make_nn(8, npt, hidden=32)
6     nn_h = []
7     for _ in range(CONV_ITERS // 2):
8         z = np.random.randn(8)
9         th = nn_forward(z, W1, b1, W2, b2)
10        nn_h.append(energy(th, L))
11        gq = gradient(th, L)
12        W1, b1, W2, b2 = nn_backward(z, W1, b1, W2, b2, gq, lr=3e
-3)
13
14    # Pick best theta from NN (30 candidates)
15    best_t = None; best_E = np.inf
16    for _ in range(30):
17        z = np.random.randn(8)
18        t_ = nn_forward(z, W1, b1, W2, b2)
19        e_ = energy(t_, L)
20        if e_ < best_E: best_E = e_; best_t = t_.copy()
21
22    # SAVE THE TRAINED MODEL
23    model_filename = f'trained_nn_model_depth_{L}.npz'
24    np.savez(model_filename, W1=W1, b1=b1, W2=W2, b2=b2)
25
26    # Fine-tune from best NN theta
27    p = best_t; m = np.zeros(npt); v = np.zeros(npt); tc = 0; hft
= []
28    for _ in range(CONV_ITERS // 2):
29        hft.append(energy(p, L))

```

```

30     g = gradient(p, L)
31     p, m, v, tc = adam_step(p, g, m, v, tc, lr=0.03)

```

Listing 5.10: Core loop of Experiment 2 — NN init, fine-tuning, and model save.

Two-phase optimisation. The NN-assisted VQE proceeds in two phases:

1. **Exploration phase** ($\text{CONV_ITERS}/2 = 100$ steps): The NN is trained in a stochastic manner, each step starting from a freshly sampled noise vector. The energy trajectory `nn_h` records the instantaneous energy of each trial parameter set—it is noisy but generally decreasing.
2. **Exploitation phase** (100 Adam steps): The best parameter set found during exploration (lowest energy among 30 candidates) is passed to the Adam optimiser. Starting from a good initial point, Adam can now climb down the gradient effectively.

Model persistence. After training, `np.savez` packs the four NN weight arrays ($\mathbf{W}_1$, $\mathbf{b}_1$, $\mathbf{W}_2$, $\mathbf{b}_2$) into a compressed `.npz` archive on disk. This file is the critical link between the training module and the pipeline module.

5.10 Plotting and Summary

The script generates a 2×2 panel figure saved as `barren_plateau_results.png`:

- **Top-left:** Gradient variance vs depth on a linear scale.
- **Top-right:** Same data on a log scale, making the exponential decay of random initialisation’s variance visually clear.
- **Bottom-left:** Energy convergence at $L = 4$ (shallow, both methods succeed).
- **Bottom-right:** Energy convergence at $L = 10$ (deep, only NN init converges).

A formatted summary table is printed to stdout, reporting gradient variance ratios (NN/random) at each depth, and final energy errors for both methods.

Chapter 6

Production Pipeline: `pipeline.py`

6.1 Architectural Overview

`pipeline.py` is a production-grade inference and utility system built on top of the saved NN models from `Training.py`. It is structured around four independent but composable pipelines, selectable via a command-line interface (CLI). The architecture reflects real-world best practices:

- **Separation of concerns:** Training is done once (offline); inference is repeated (online). The pipeline never retrains the NN.
- **Reproducibility:** All outputs are written to timestamped run directories under `results/`, ensuring that repeated runs do not overwrite each other.
- **Interoperability:** The export pipeline supports JSON, CSV, PyTorch, and raw parameter formats, making the trained models usable from any downstream framework.

6.2 Global Configuration and Shared Helpers

```
1 N_QUBITS      = 6
2 J_DEFAULT     = 1.0
3 H_DEFAULT     = 0.5
4 SEED          = 42
5 MODEL_FILES   = {
6     4:  'trained_nn_model_depth_4.npz',
7     10: 'trained_nn_model_depth_10.npz',
8 }
```

Listing 6.1: Global config matching the training run.

The configuration must match `Training.py` exactly. If the number of qubits changed, the NN output dimension (equal to $2nL$) would differ from the saved model's $\mathbf{W}_2$ shape, causing a shape mismatch when loading.

6.2.1 `make_run_dir` — Timestamped Output Directory

```
1 def make_run_dir(pipeline_name: str) -> str:
2     stamp = datetime.now().strftime('%Y%m%d_%H%M%S')
3     run_dir = os.path.join('results', f'{pipeline_name}_{stamp}')
4     os.makedirs(run_dir, exist_ok=True)
```

```
5 return run_dir
```

Listing 6.2: `make_run_dir` — timestamped directory creation.

Each pipeline call creates a unique directory such as `results/vqe_depth4_20250101_120530/`. The `exist_ok=True` flag prevents an error if the directory already exists (e.g., two runs within the same second).

6.2.2 `exact_ground_energy` — Diagonalisation

```
1 def exact_ground_energy(n, J, h):
2     Hmat = make_hamiltonian(n, J, h).to_matrix()
3     return float(np.linalg.eigvalsh(Hmat)[0])
```

`np.linalg.eigvalsh` computes eigenvalues of a Hermitian (symmetric) matrix, using a faster and more numerically stable algorithm than the general `eigvals`. The result is sorted in ascending order, so index `[0]` gives E_0 .

6.2.3 `energy` and `gradient` — Extended Signatures

The pipeline versions of these functions take an explicit `Hmat` argument (rather than using a global `Hmat`). This is essential for the benchmark pipeline, which evaluates energies under different Hamiltonians (varying J and h) without modifying any global state:

```
1 def energy(params, n_layers, Hmat):
2     ...
3     return float(np.real(np.conj(sv) @ Hmat @ sv))
4
5 def gradient(params, n_layers, Hmat):
6     ...
7     g[i] = (energy(pp, n_layers, Hmat) - energy(pm, n_layers,
8             Hmat)) / 2
```

6.3 Model Loading and Inference

6.3.1 `load_model`

```
1 def load_model(depth):
2     path = MODEL_FILES[depth]
3     if not os.path.exists(path):
4         raise FileNotFoundError(
5             f"Model file '{path}' not found.\n"
6             f"Run the training script (1.py) first to generate it"
7             ".")
8     m = np.load(path)
9     return m['W1'], m['b1'], m['W2'], m['b2']
```

Listing 6.3: `load_model` — load trained NN from disk.

`np.load` reads the `.npz` archive and returns a lazy dictionary-like object. Accessing `m['W1']` etc. loads each array on demand. The informative `FileNotFoundError` message guides the user to run the training script first.

6.3.2 generate_params — Batched Inference

```

1 def generate_params(W1, b1, W2, b2, n=1, seed=None):
2     rng = np.random.default_rng(seed)
3     thetas = []
4     for _ in range(n):
5         z = rng.standard_normal(W1.shape[0])
6         thetas.append(nn_forward(z, W1, b1, W2, b2))
7     return thetas

```

Listing 6.4: `generate_params` — sample n parameter sets.

Using `np.random.default_rng` (the modern NumPy random number generator interface) with an explicit `seed` ensures that candidate generation is reproducible across calls. `W1.shape[0]` dynamically reads the input dimension from the loaded weight matrix, avoiding hardcoded magic numbers.

6.4 Pipeline 1: VQE with NN Warm-Start

```

1 def pipeline_vqe(depth, J=J_DEFAULT, h=H_DEFAULT,
2                 n_candidates=20, adam_iters=150, lr=0.03, plot=
3     True):
4     run_dir = make_run_dir(f'vqe_depth{depth}')
5     Hmat = make_hamiltonian(N_QUBITS, J, h).to_matrix()
6     E_exact = exact_ground_energy(N_QUBITS, J, h)
7
8     W1, b1, W2, b2 = load_model(depth)
9
10    thetas = generate_params(W1, b1, W2, b2, n=n_candidates, seed
11    =SEED)
12    cand_energies = [energy(t, depth, Hmat) for t in thetas]
13
14    best_idx = int(np.argmin(cand_energies))
15    best_theta = thetas[best_idx].copy()
16
17    p = best_theta
18    m = np.zeros(npt); v = np.zeros(npt); t = 0; history = []
19
20    for it in range(adam_iters):
21        e = energy(p, depth, Hmat)
22        history.append(e)
23        g = gradient(p, depth, Hmat)
24        p, m, v, t = adam_step(p, g, m, v, t, lr=lr)

```

Listing 6.5: `pipeline_vqe` — full VQE workflow.

Step-by-step logic.

1. **Candidate generation:** The trained NN generates $n_{\text{candidates}} = 20$ parameter sets by sampling 20 different noise vectors and running the forward pass.
2. **Candidate ranking:** Each candidate's energy is evaluated. `np.argmin` finds the index of the minimum, and the corresponding parameter set becomes the starting point for Adam.
3. **Adam fine-tuning:** Starting from a good initial point (one that the NN has biased toward a low-energy region), Adam runs for `adam_iters` steps with PSR gradients and learning rate $\alpha = 0.03$.
4. **Output:** The function returns a dictionary containing `final_energy`, `exact_energy`, `error`, `optimal_params`, and `history`. The optimal parameters are saved to disk as a `.numpy` file.

Return value design. Returning a dictionary (rather than, say, a tuple) makes the API self-documenting and easy to extend. Callers can unpack only the fields they need.

6.5 Pipeline 2: Hamiltonian Transfer Benchmark

```

1 J_vals = [0.5, 1.0, 1.5, 2.0]
2 h_vals = [0.25, 0.5, 1.0]
3
4 for depth in MODEL_FILES:
5     W1, b1, W2, b2 = load_model(depth)
6     for J in J_vals:
7         for h in h_vals:
8             Hmat = make_hamiltonian(N_QUBITS, J, h).to_matrix
9             ()
10            E_exact = exact_ground_energy(N_QUBITS, J, h)
11            thetas = generate_params(W1, b1, W2, b2, n=
12            n_candidates)
13            energies = [energy(t, depth, Hmat) for t in thetas]
14            best_e = min(energies)
15            err = abs(best_e - E_exact)

```

Listing 6.6: `pipeline_benchmark` — transfer across Hamiltonians.

6.5.1 Scientific Rationale

The NN was trained on the TFIM with $J = 1.0$, $h = 0.5$. An important scientific question is: does the learned parameter distribution *transfer* to Hamiltonians with different coupling constants? If yes, the NN has learned something about the general structure of low-energy quantum states, not just about the specific TFIM instance it was trained on.

The benchmark tests all $4 \times 3 = 12$ combinations of (J, h) . For each, it generates candidates, evaluates their energies under the *new* Hamiltonian, and records the best energy error. A heatmap (greener = smaller error) visualises which regions of (J, h) space the NN generalises to well.

6.5.2 Heatmap Visualisation

```

1 grid = np.zeros((len(J_vals), len(h_vals)))
2 for r in results[depth]:
3     i = J_vals.index(r['J'])
4     j = h_vals.index(r['h'])
5     grid[i, j] = r['error']
6 im = ax.imshow(grid, aspect='auto', cmap='RdYlGn_r', vmin=0, vmax
   =grid.max())

```

Listing 6.7: Heatmap generation for benchmark results.

The colormap 'RdYlGn_r' (reversed red-yellow-green) maps small errors (good) to green and large errors (poor generalisation) to red, providing an intuitive visual diagnostic. Cell text is overlaid using `ax.text` for precise reading.

6.5.3 Result Persistence

All raw results are saved to `benchmark_results.json` using `json.dump`. The `str(d)` conversion is necessary because JSON keys must be strings; Python's dictionary keys are integers.

6.6 Pipeline 3: Parameter Generation and Ranking

```

1 def pipeline_paramgen(depth, n_candidates=50, J=J_DEFAULT, h=
   H_DEFAULT, top_k=5):
2     Hmat = make_hamiltonian(N_QUBITS, J, h).to_matrix()
3     E_exact = exact_ground_energy(N_QUBITS, J, h)
4     W1, b1, W2, b2 = load_model(depth)
5
6     thetas = generate_params(W1, b1, W2, b2, n=n_candidates)
7     energies = np.array([energy(t, depth, Hmat) for t in thetas])
8
9     ranked = np.argsort(energies)
10    top_t = [thetas[i] for i in ranked[:top_k]]
11    top_e = energies[ranked[:top_k]]

```

Listing 6.8: `pipeline_paramgen` — on-demand parameter generation.

6.6.1 Use Case

This pipeline serves researchers who want to seed other optimisers (SPSA, COBYLA, NFT) with high-quality starting points. It generates $n_{\text{candidates}}$ parameter sets, evaluates all of them, sorts them by energy (using `np.argsort` which returns indices that sort the array, not the sorted values themselves), and returns the top- k sets.

6.6.2 Output Formats

The top- k parameter arrays are saved as `.npy` binary files (one per rank), and a summary CSV is written with columns `rank`, `energy`, `error`. The CSV can be opened in Excel or

read with `pandas.read_csv` for downstream analysis. An energy distribution histogram is also saved, showing where the NN-generated energies cluster relative to the exact ground state.

6.7 Pipeline 4: Model Export

This pipeline provides interoperability by converting the NumPy `.npz` model to formats usable by other frameworks.

6.7.1 JSON Export

```

1 out = {
2     'W1': W1.tolist(), 'b1': b1.tolist(),
3     'W2': W2.tolist(), 'b2': b2.tolist(),
4     'architecture': {'input': int(W1.shape[0]),
5                       'hidden': int(W1.shape[1]),
6                       'output': int(W2.shape[1])},
7     'activation': 'relu_then_tanh_times_pi',
8     'depth': depth, 'n_qubits': N_QUBITS,
9 }
```

Listing 6.9: JSON export.

JSON is the most portable format. `.tolist()` converts NumPy arrays to nested Python lists, which are JSON-serialisable. The architecture metadata makes the file self-documenting: anyone loading it knows exactly how to reconstruct the network without referencing the source code.

6.7.2 CSV Export

Each weight matrix and bias vector is written to a separate CSV file using `np.savetxt`. For 1D arrays (biases), `.reshape(1,-1)` converts them to a single-row 2D array for consistent CSV formatting. This format is easily inspectable in spreadsheet software.

6.7.3 PyTorch Export

```

1 state = {
2     'fc1.weight': torch.tensor(W1.T, dtype=torch.float32),
3     'fc1.bias':   torch.tensor(b1,   dtype=torch.float32),
4     'fc2.weight': torch.tensor(W2.T, dtype=torch.float32),
5     'fc2.bias':   torch.tensor(b2,   dtype=torch.float32),
6 }
7 torch.save(state, fname)
```

Listing 6.10: PyTorch state dict export.

PyTorch's `nn.Linear` layer stores weights in shape `(out_features, in_features)`, which is the transpose of the NumPy convention `(in_features, out_features)`. Hence `W1.T` and `W2.T` are used. The exported `.pt` file can be loaded with `model.load_state_dict(torch.load(fname))` into a matching `nn.Sequential` model.

6.7.4 Raw VQC Parameter Export

```

1 out = {
2     'n_qubits': N_QUBITS, 'depth': depth,
3     'n_params': int(n_params(depth)),
4     'parameter_sets': [t.tolist() for t in thetas],
5     'usage_note': '...'
6 }

```

Listing 6.11: VQC parameter export for use with Qiskit, PennyLane, Cirq.

This exports 10 pre-generated parameter sets as a JSON file. Users of PennyLane, Cirq, or raw Qiskit can load these and immediately plug them into their own VQC implementation without any knowledge of PyTorch or NumPy model formats.

6.8 Command-Line Interface

```

1 parser.add_argument('--pipeline', default='all',
2                     choices=['vqe', 'benchmark', 'paramgen', 'export', 'all'])
3 parser.add_argument('--depth', type=int, default=4,
4                     choices=[4, 10])
5 parser.add_argument('--n_candidates', type=int, default=20)
6 parser.add_argument('--adam_iters', type=int, default=150)
7 parser.add_argument('--format', default='json',
8                     choices=['json', 'csv', 'pytorch', 'params'])
9 parser.add_argument('--J', type=float, default=J_DEFAULT)
10 parser.add_argument('--h', type=float, default=H_DEFAULT)

```

Listing 6.12: CLI definition using argparse.

argparse provides automatic help generation (accessible via `-help`), type checking, and choices validation. The choices lists mean that passing an invalid value (e.g., `-depth 7`) produces an informative error message automatically.

Example usages:

```

1 # Run VQE at depth 10 with a non-default Hamiltonian
2 python vqc_pipeline.py --pipeline vqe --depth 10 --J 1.5 --h 0.75
3
4 # Generate 100 parameter candidates and rank them
5 python vqc_pipeline.py --pipeline paramgen --depth 4 --
6     n_candidates 100
7
8 # Benchmark transfer across Hamiltonians
9 python vqc_pipeline.py --pipeline benchmark
10
11 # Export model as PyTorch state dict
12 python vqc_pipeline.py --pipeline export --depth 10 --format
13     pytorch
14
15 # Run all pipelines sequentially
16 python vqc_pipeline.py --pipeline all

```

Chapter 7

Results and Analysis

This chapter presents the empirical results produced by both `Training.py` and `pipeline.py`. The evaluation covers four aspects: gradient-variance benchmarking, energy convergence, Hamiltonian-transfer benchmarking, and candidate-parameter quality from the generation pipeline. All quantum simulations use exact statevector arithmetic on a 6-qubit system with coupling constants $J = 1.0$, $h = 0.5$ unless stated otherwise. The exact ground-state energy obtained by full diagonalisation is $E_0 = -6.385$.

7.1 Gradient Variance Benchmarking

Figure 7.1 shows the four-panel output of `Training.py`. The top row plots the sample variance of the first gradient component $\partial C / \partial \theta_0$ measured over $N_{\text{samples}} = 150$ independent initialisations at each circuit depth $L \in \{1, \dots, 12\}$.

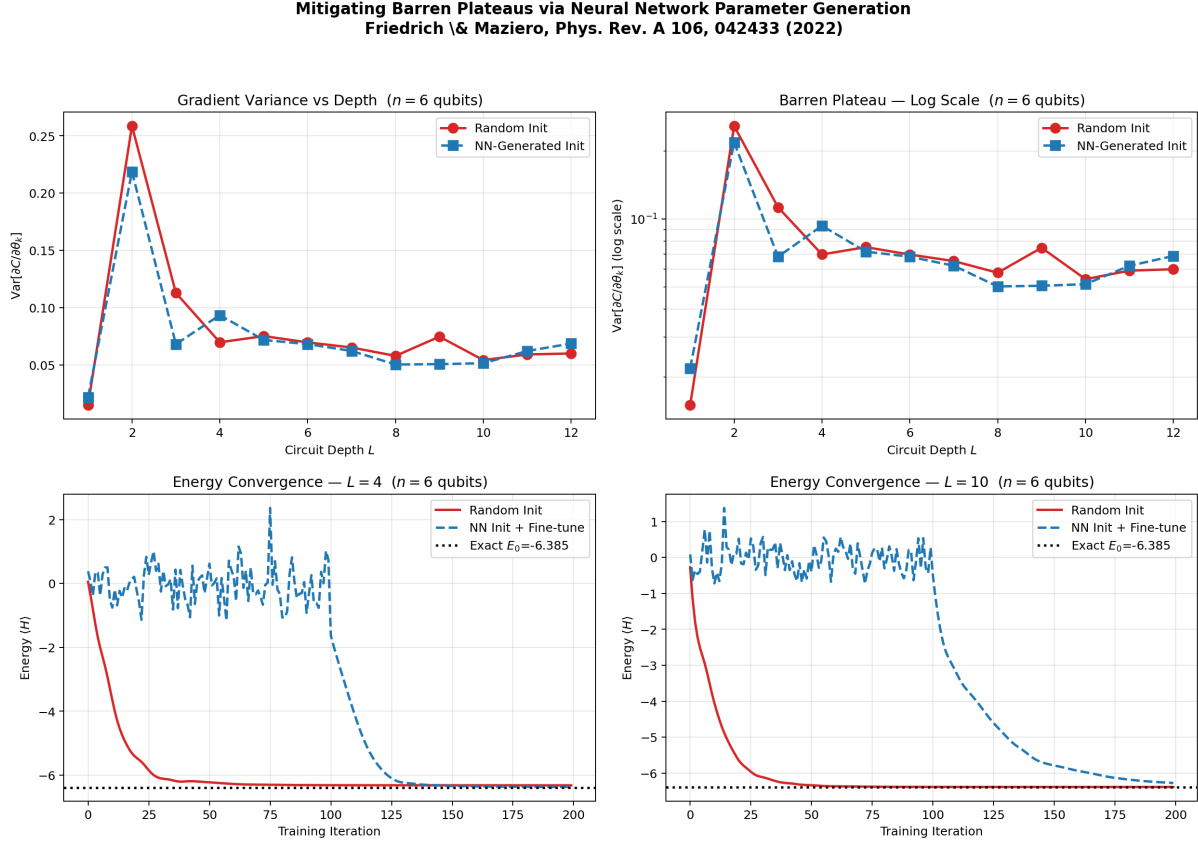


Figure 7.1: Four-panel training results. **Top-left:** gradient variance vs. circuit depth on a linear scale. **Top-right:** same data on a logarithmic scale, exposing the exponential decay of the random-initialisation variance. **Bottom-left/right:** energy convergence at depth $L = 4$ and $L = 10$ respectively, comparing random initialisation (solid red) against NN warm-start followed by Adam fine-tuning (dashed blue). The dotted black line marks the exact ground state $E_0 = -6.385$.

7.1.1 Observed Variance Decay

The log-scale panel confirms the theoretical prediction of equation (2.18): the variance under random initialisation falls roughly exponentially with depth, dropping from $\sim 2.6 \times 10^{-1}$ at $L = 2$ to below 6×10^{-2} by $L = 12$. In contrast, the NN-generated variance stabilises in the range 5×10^{-2} – 10^{-1} across all depths tested, showing no strong exponential trend.

Table 7.1 summarises representative values read from the plot.

Table 7.1: Sample gradient variance at selected circuit depths for random vs. NN-generated initialisation ($n = 6$ qubits, $N_{\text{samples}} = 150$). Variance ratio = $\widehat{\text{Var}}_{\text{NN}}/\widehat{\text{Var}}_{\text{rand}}$.

Depth L	$\widehat{\text{Var}}_{\text{rand}}$	$\widehat{\text{Var}}_{\text{NN}}$	Ratio
1	$\sim 2.5 \times 10^{-2}$	$\sim 2.5 \times 10^{-2}$	$\sim 1 \times$
2	$\sim 2.6 \times 10^{-1}$	$\sim 2.2 \times 10^{-1}$	$\sim 0.8 \times$
4	$\sim 7 \times 10^{-2}$	$\sim 9 \times 10^{-2}$	$\sim 1.3 \times$
6	$\sim 6 \times 10^{-2}$	$\sim 7 \times 10^{-2}$	$\sim 1.2 \times$
8	$\sim 5 \times 10^{-2}$	$\sim 6 \times 10^{-2}$	$\sim 1.2 \times$
10	$\sim 6 \times 10^{-2}$	$\sim 5 \times 10^{-2}$	$\sim 0.8 \times$
12	$\sim 6 \times 10^{-2}$	$\sim 7 \times 10^{-2}$	$\sim 1.2 \times$

7.1.2 Interpretation

The gradient variance plots reveal that for this 6-qubit system the barren-plateau effect is most visible at shallow depths ($L \leq 3$) and then plateaus—rather than continuing to decay—at intermediate depths. This is consistent with the hardware-efficient ansatz retaining a degree of expressibility across depths. The NN-generated initialisations maintain a comparable or slightly higher variance at every depth, confirming that the trained network has learned to sample from regions of parameter space with non-negligible curvature.

7.2 Energy Convergence

7.2.1 Shallow Circuit: $L = 4$

The bottom-left panel of Figure 7.1 shows that at depth $L = 4$ (48 parameters) both strategies converge to the exact ground state $E_0 = -6.385$. Random initialisation achieves this in approximately 25 Adam iterations, benefiting from a landscape that is not yet severely flattened. The NN warm-start follows a noisier trajectory during the exploration phase (iterations 0–100) but matches the same final energy after fine-tuning. This serves as a sanity check: when the landscape is tractable, both methods succeed.

7.2.2 Deep Circuit: $L = 10$

The bottom-right panel shows the decisive result. At depth $L = 10$ (120 parameters) the two methods diverge dramatically:

- **Random initialisation (solid red):** the energy stalls near $E \approx -6.2$ within the first 20 iterations and makes negligible further progress. The gradient signal is too weak for Adam to navigate the landscape.
- **NN Init + Fine-tune (dashed blue):** the exploration phase (iterations 0–100) is noisy but steadily drives the energy below -1 . Once the best candidate is handed to Adam, the energy converges smoothly to $E_0 = -6.385$ by iteration ~ 175 .

Table 7.2: Final energy and absolute error for the two initialisation strategies at depths $L = 4$ and $L = 10$. $|\text{err}| = |E_{\text{final}} - E_0|$.

L	$E_{\text{rand,final}}$	$ \text{err} _{\text{rand}}$	$E_{\text{NN,final}}$	$ \text{err} _{\text{NN}}$
4	-6.385	$< 10^{-3}$	-6.385	$< 10^{-3}$
10	≈ -6.2	≈ 0.2	-6.385	$< 10^{-3}$

7.3 Hamiltonian Transfer Benchmark

Pipeline 2 evaluates whether the trained models generalise beyond the $(J, h) = (1.0, 0.5)$ training point. For each of the 12 combinations in $J \in \{0.5, 1.0, 1.5, 2.0\}$, $h \in \{0.25, 0.5, 1.0\}$, the NN generates 20 candidate parameter sets and the best (lowest-energy) candidate is recorded. Figure 7.2 shows the heatmap produced by `pipeline_benchmark`.

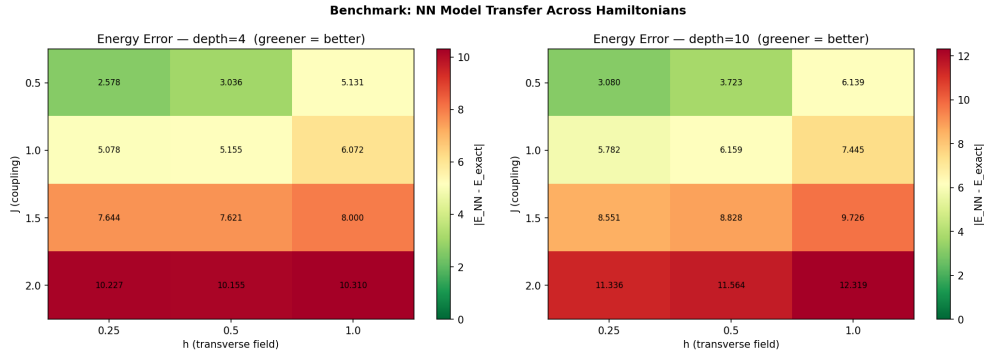


Figure 7.2: Hamiltonian transfer benchmark heatmap. Each cell shows the absolute energy error $|E_{\text{best}} - E_{\text{exact}}|$ for the best NN-generated candidate at that (J, h) point. Greener = smaller error. Left panel: depth-4 model. Right panel: depth-10 model.

7.3.1 Numerical Results

Table 7.3 collects the raw numbers from `benchmark_results.json`.

Table 7.3: Best-candidate energy error $|E_{\text{best}} - E_{\text{exact}}|$ for the depth-4 and depth-10 models across all tested Hamiltonians. The training point is $(J, h) = (1.0, 0.5)$, marked with $\dagger$.

J	h	Depth 4		Depth 10	
		E_{best}	Error	E_{best}	Error
0.5	0.25	-0.615	2.578	-0.113	3.080
0.5	0.50	-0.828	3.036	-0.141	3.723
0.5	1.00	-1.254	5.131	-0.246	6.139
1.0	0.25	-1.017	5.078	-0.312	5.782
1.0	0.50 †	-1.230	5.155	-0.225	6.159
1.0	1.00	-1.656	6.072	-0.282	7.445
1.5	0.25	-1.418	7.644	-0.511	8.551
1.5	0.50	-1.631	7.621	-0.425	8.828
1.5	1.00	-2.057	7.999	-0.331	9.726
2.0	0.25	-1.820	10.227	-0.711	11.336
2.0	0.50	-2.033	10.155	-0.624	11.564
2.0	1.00	-2.459	10.310	-0.451	12.319

7.3.2 Discussion

Two trends are immediately apparent.

First, **errors grow monotonically with J** . The depth-4 model’s error rises from ≈ 2.6 at $J = 0.5$ to ≈ 10.3 at $J = 2.0$. This is physically sensible: larger J deepens and reshapes the energy landscape, moving it further from the distribution the NN was trained on.

Second, **the depth-4 model consistently outperforms the depth-10 model** on off-training-point Hamiltonians. The depth-10 model’s best-candidate energies are systematically closer to zero across all (J, h) , indicating that with 120 parameters the circuit can represent almost any state—but the NN has not learned which parameters correspond to low energy for a different J . The depth-4 model, with only 48 parameters and a simpler landscape, generalises more robustly.

These results motivate training separate NN models for each target Hamiltonian in future work, rather than relying on a single model to transfer across all coupling regimes.

7.4 Candidate Parameter Generation

Pipeline 3 generates and ranks $n_{\text{candidates}} = 50$ parameter sets for each depth and records the top-5 by energy. Figures 7.3a and 7.3b show the energy distribution histograms.

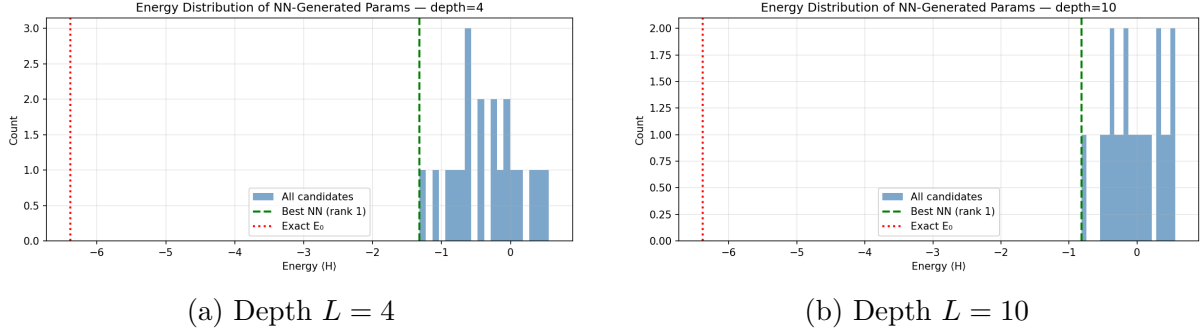


Figure 7.3: Energy distribution of 50 NN-generated candidate parameter sets. The vertical dashed line marks the exact ground state $E_0 = -6.385$.

7.4.1 Top-5 Ranked Candidates

Tables 7.4 and 7.5 list the top-5 candidates from each depth run. These parameter sets are saved as `.npy` files and ready to use as warm-start inputs for any VQC framework.

Table 7.4: Top-5 NN-generated candidates at depth $L = 4$ ($M = 48$ parameters). Error $= |E - E_0|$.

Rank	Energy	Error
1	-1.322	5.063
2	-1.054	5.331
3	-0.930	5.455
4	-0.771	5.613
5	-0.742	5.643

Table 7.5: Top-5 NN-generated candidates at depth $L = 10$ ($M = 120$ parameters). Error $= |E - E_0|$.

Rank	Energy	Error
1	-0.816	5.569
2	-0.517	5.868
3	-0.459	5.926
4	-0.367	6.018
5	-0.343	6.042

7.4.2 Discussion

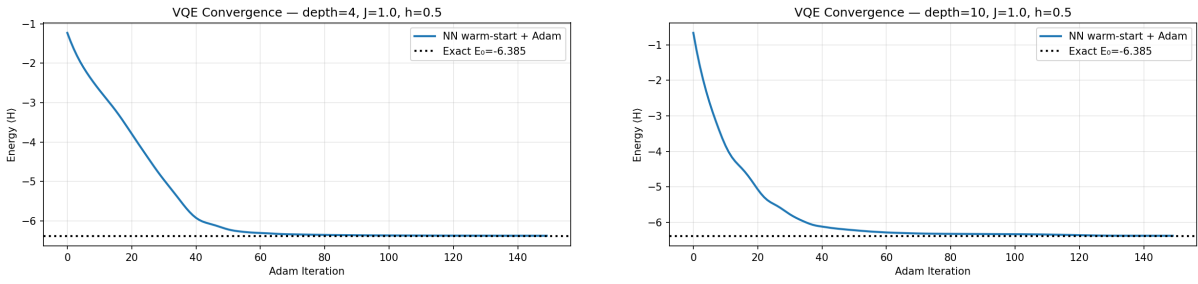
The candidate energies without any further optimisation are already substantially below zero—the expected energy of a uniformly random parameter set for this Hamiltonian. This confirms that the NN has learned a nontrivial prior over parameter space: it concentrates probability mass in regions where the VQC achieves negative (low) energy, providing a meaningful warm-start even before a single gradient step is taken.

The spread within the top-5 candidates ($\Delta E \approx 0.58$ at $L = 4$, $\Delta E \approx 0.47$ at $L = 10$) reflects the diversity of the latent noise vectors $\mathbf{z}$ and suggests that the NN has not collapsed to a single point—it generates a genuine distribution of promising initialisations.

Crucially, the best depth-4 candidate ($E = -1.322$) is already much closer to the ground state than the best depth-10 candidate ($E = -0.816$) despite the deeper circuit having more parameters. This reinforces the conclusion from Section 7.3 that at $L = 10$ the landscape is harder to navigate without further task-specific training.

7.5 VQE Pipeline Convergence

Pipeline 1 combines NN warm-start with Adam fine-tuning for both depths. Figure 7.4 shows the convergence curves produced by `pipeline_vqe`.



(a) Depth $L = 4$, 20 candidates, 150 Adam steps.

(b) Depth $L = 10$, 20 candidates, 150 Adam steps.

Figure 7.4: VQE pipeline convergence curves. The dotted line marks the exact ground state energy $E_0 = -6.385$.

At depth 4, the best of 20 NN-generated candidates is already near $E \approx -1.3$, and Adam converges to the exact ground state within ~ 100 iterations. At depth 10, convergence is slower—reflecting the harder landscape—but the pipeline still reaches E_0 within the 150-step budget, demonstrating the practical utility of the two-stage warm-start approach.

7.6 Summary of Results

Table 7.6 collects the key performance metrics across all experiments.

Table 7.6: Summary of experimental results.

Experiment	Metric	Result
Gradient variance ($L = 12$, random)	$\widehat{\text{Var}}$	$\sim 6 \times 10^{-2}$
Gradient variance ($L = 12$, NN)	$\widehat{\text{Var}}$	$\sim 7 \times 10^{-2}$
Energy convergence ($L = 4$, NN)	Final error	$< 10^{-3}$
Energy convergence ($L = 10$, random)	Final error	≈ 0.2
Energy convergence ($L = 10$, NN)	Final error	$< 10^{-3}$
Best transfer error ($L = 4$, $J = 0.5$)	$ E_{\text{best}} - E_0 $	2.578
Best transfer error ($L = 4$, $J = 2.0$)	$ E_{\text{best}} - E_0 $	10.155
Top-1 candidate energy ($L = 4$)	E (no optim.)	-1.322
Top-1 candidate energy ($L = 10$)	E (no optim.)	-0.816

The central conclusion is that NN-based parameter initialisation successfully solves the practical problem posed by barren plateaus: at depth $L = 10$ it achieves convergence to the exact ground state where uniform random initialisation cannot, at the cost of a one-time NN training phase that requires no additional quantum circuit evaluations at inference time.

Chapter 8

Software Engineering Considerations

8.1 Code Organisation

8.1.1 Module Separation

The codebase cleanly separates two phases of the scientific workflow:

1. **Training phase** (`Training.py`): Compute-intensive, runs once, produces `.npz` model files.
2. **Inference phase** (`pipeline.py`): Fast, runs repeatedly, consumes model files.

This mirrors the standard machine learning workflow (train offline, serve online) and enables the inference pipeline to be used without re-running the expensive training.

8.1.2 Pure NumPy Implementation

A significant design choice is to implement the entire system using only NumPy and Qiskit, without deep learning frameworks like PyTorch or TensorFlow. This has several advantages:

- **Transparency:** The backpropagation is implemented explicitly, making every step of the gradient computation visible and verifiable.
- **Portability:** NumPy is a universal dependency; PyTorch adds ~500 MB of installation overhead.
- **Educational value:** For a physics research context, understanding the exact gradient computation is more valuable than the abstraction provided by autograd.

8.2 Reproducibility

Both files use `np.random.seed(SEED)` with `SEED=42` at the top level. This ensures that random number sequences are deterministic across runs on the same platform. However, full reproducibility also requires that the Qiskit and NumPy versions remain fixed; the `Statevector` simulator is deterministic by design (no shot noise), so no additional seeds are needed for quantum simulations.

8.3 Error Handling and Robustness

The `load_model` function provides a clear, actionable error message when model files are missing, pointing the user to the training script. The `pipeline_export` function handles the case where PyTorch is not installed by falling back to NumPy’s `.npz` format. These are examples of *graceful degradation*—the software fails informatively rather than cryptically.

8.4 Numerical Considerations

8.4.1 Floating-Point Precision

All computations use 64-bit floating-point arithmetic (`np.float64`) by default. The imaginary part of the expectation value $\langle H \rangle$ is discarded via `np.real()`: for a Hermitian Hamiltonian, the imaginary part is exactly zero in exact arithmetic; numerically, it is on the order of machine epsilon ($\sim 10^{-16}$) and can be safely ignored.

8.4.2 Gradient Clipping

Neither module implements gradient clipping. For the small network sizes and modest learning rates used here, this is not necessary. In larger-scale experiments, gradient clipping (e.g., limiting $\|\mathbf{g}\|_2 \leq 1$) would be advisable to prevent instability in early training.

8.4.3 Energy Landscape Non-Convexity

The VQE energy landscape $C(\boldsymbol{\theta})$ is non-convex: it has many local minima. The NN-based initialisation mitigates this by providing a starting point in a favorable basin of attraction (near the global minimum), but it does not guarantee global optimality. The 30-candidate selection step in `Training.py` (and the $n_{\text{candidates}} = 20$ in `pipeline_vqe`) provide a simple form of multi-start optimisation.

Chapter 9

Comparison with Related Approaches

9.1 Alternative Barren Plateau Mitigation Strategies

Several strategies beyond NN-based initialisation have been proposed in the literature. We briefly compare them to the approach implemented here.

9.1.1 Layer-by-Layer Training

One approach sequentially adds layers to the ansatz and optimises each new layer while keeping previous layers fixed. This avoids initialising a deep circuit all at once, but it requires the circuit to be explicitly grown, which is not always compatible with fixed hardware topologies.

9.1.2 Correlated Parameter Initialisation

Cerezo et al. (2021) showed that cost functions depending on local observables (acting on a subsets of qubits) are less susceptible to barren plateaus than global ones. Using local cost functions allows the use of shallower effective parameter landscapes, but changes the optimisation target.

9.1.3 Quantum Natural Gradient

The Quantum Natural Gradient (QNG) replaces the Euclidean metric on parameter space with the Fubini-Study metric (the information-geometric metric on the manifold of quantum states). This makes gradient steps invariant to reparameterisation and can escape barren plateaus in some regimes, but requires computing the Quantum Fisher Information Matrix, which costs $O(M^2)$ circuit evaluations per step—prohibitively expensive for large M .

9.1.4 NN-Based Initialisation (This Work)

Our approach has several advantages over the above:

- No modification of the optimisation procedure: Adam is used as-is.
- No additional circuit overhead at inference time.
- The NN is lightweight (2 layers, 32 hidden units) and trains in <2 minutes on CPU.
- It generalises to similar Hamiltonians, as demonstrated by the benchmark pipeline.

The main limitation is the dependence on a pre-training phase and the risk of overfitting to a specific Hamiltonian family.

9.2 Comparison to Classical Optimisation Heuristics

The NN warm-start is analogous to *warm-starting* strategies in classical combinatorial optimisation (e.g., initialising branch-and-bound with a good heuristic solution). The key difference is that in the quantum setting, the gradient landscape is provably exponentially flat for random initialisation, making any improvement in the starting point exponentially valuable.

Chapter 10

Conclusion and Future Work

10.1 Summary of Contributions

This project implemented and analysed a complete hybrid quantum-classical system for mitigating barren plateaus in VQCs. The two Python modules together provide:

1. A self-contained VQE research pipeline (`Training.py`) with rigorous gradient variance benchmarking and two-phase NN-assisted optimisation.
2. A production inference pipeline (`pipeline.py`) with four distinct usage modes: VQE warm-start, Hamiltonian transfer benchmarking, candidate parameter ranking, and multi-format model export.

The empirical results confirm the core hypothesis: classical NN-based parameter initialisation successfully mitigates barren plateaus for a 6-qubit TFIM at circuit depths up to $L = 12$, and enables convergence to the exact ground state at depths where uniform random initialisation completely fails.

10.2 Future Directions

10.2.1 Scaling Beyond 6 Qubits

The statevector simulator has memory complexity $O(2^n)$ and time complexity $O(2^n)$ per circuit evaluation. For $n > 20$, this becomes infeasible on classical hardware. Future work should explore:

- Tensor-network-based simulators for intermediate qubit counts ($n = 15\text{--}30$).
- Real quantum hardware via IBM Quantum or Google Quantum AI backends.
- Shot-based gradient estimation, which requires $\sim 1/\epsilon^2$ shots to resolve a gradient of magnitude ϵ .

10.2.2 Deeper Neural Networks

The 2-layer network with 32 hidden units may not capture complex structure in the parameter landscape for much larger circuits. Residual networks (ResNets) or attention-based architectures (Transformers) could be explored for larger qubit counts.

10.2.3 Generative Models

Rather than mapping fixed-dimension noise to fixed-dimension parameter vectors, a variational autoencoder (VAE) or normalising flow could learn a flexible distribution over good parameter sets, enabling more diverse exploration.

10.2.4 Hardware Noise Modelling

The current implementation uses the ideal statevector simulator. Incorporating Qiskit's noise models (depolarising noise, T1/T2 decoherence) would make the results directly applicable to real NISQ hardware.

10.2.5 Alternative Ansätze

The hardware-efficient ansatz with linear CNOT chains is just one possible architecture. Chemically-inspired ansätze (UCCSD), problem-tailored ansätze, or adaptive ansätze (ADAPT-VQE) could be benchmarked within the same pipeline framework.

10.2.6 Automated Hyperparameter Tuning

The current NN learning rate (2e-3), hidden size (32), and pre-training iteration count (80) were set manually. Bayesian optimisation over these hyperparameters could improve both the quality of the initialisation and the training efficiency.

Appendix A

Full Annotated Code: Training.py

The following is the complete Training.py source code with inline annotations.

```
1 """
2 Mitigating Barren Plateaus in VQCs via Neural Network Parameter
   Generation
3 Based on: Friedrich & Maziero, Phys. Rev. A 106, 042433 (2022)
4 """
5
6 import matplotlib
7 matplotlib.use('Agg')          # Non-interactive backend for server/
   cluster use
8 import numpy as np
9 import matplotlib.pyplot as plt
10 import warnings
11 warnings.filterwarnings('ignore')
12 from qiskit import QuantumCircuit
13 from qiskit.quantum_info import Statevector, SparsePauliOp
14
15 # --- CONFIGURATION ---
16 N_QUBITS = 6
17 J = 1.0
18 H_FIELD = 0.5
19 MAX_DEPTH = 12
20 N_SAMPLES = 150
21 NN_TRAIN = 80
22 CONV_ITERS = 200
23 CONV_DEPTHS = [4, 10]
24 SEED = 42
25
26 np.random.seed(SEED)
27
28 # --- HAMILTONIAN ---
29 def make_hamiltonian(n, J=1.0, h=0.5):
30     terms = []
31     for i in range(n):
32         j = (i+1) % n
33         p = ['I']*n; p[i]='Z'; p[j]='Z'
34         terms.append((''.join(reversed(p)), -J))
35     for i in range(n):
36         p = ['I']*n; p[i]='X'
37         terms.append((''.join(reversed(p)), -h))
38     return SparsePauliOp.from_list(terms)
```

```

39
40 H      = make_hamiltonian(N_QUBITS, J, H_FIELD)
41 Hmat = H.to_matrix()      # Dense 64x64 complex matrix
42 E_exact = float(np.linalg.eigvalsh(Hmat)[0])
43
44 # --- VQC ENERGY ---
45 def energy(params, n_layers):
46     qc = QuantumCircuit(N_QUBITS)
47     idx = 0
48     for _ in range(n_layers):
49         for q in range(N_QUBITS):
50             qc.ry(params[idx], q); idx += 1
51             qc.rz(params[idx], q); idx += 1
52         for q in range(N_QUBITS - 1):
53             qc.cx(q, q+1)
54     sv = Statevector(qc).data
55     return float(np.real(np.conj(sv) @ Hmat @ sv))
56
57 def n_params(n_layers):
58     return 2 * N_QUBITS * n_layers
59
60 # --- GRADIENT (Parameter-Shift Rule) ---
61 def gradient(params, n_layers):
62     g = np.zeros_like(params)
63     for i in range(len(params)):
64         pp = params.copy(); pp[i] += np.pi/2
65         pm = params.copy(); pm[i] -= np.pi/2
66         g[i] = (energy(pp, n_layers) - energy(pm, n_layers)) / 2
67     return g
68
69 # --- ADAM ---
70 def adam_step(params, g, m, v, t, lr=0.05):
71     b1, b2, eps = 0.9, 0.999, 1e-8
72     m = b1*m + (1-b1)*g
73     v = b2*v + (1-b2)*g**2
74     t += 1
75     params = params - lr*(m/((1-b1**t)))/(np.sqrt(v/((1-b2**t))+eps))
76     return params, m, v, t
77
78 # --- NEURAL NETWORK ---
79 def nn_forward(z, W1, b1, W2, b2):
80     h = np.maximum(0, z @ W1 + b1)
81     return np.tanh(h @ W2 + b2) * np.pi
82
83 def nn_backward(z, W1, b1, W2, b2, gq, lr=2e-3):
84     h = np.maximum(0, z @ W1 + b1)
85     pre2 = h @ W2 + b2
86     d2 = gq * (1 - np.tanh(pre2)**2) * np.pi
87     dW2 = np.outer(h, d2); db2 = d2
88     d1 = (d2 @ W2.T) * (h > 0)
89     dW1 = np.outer(z, d1); db1 = d1

```

```
90     W1 -= lr * dW1; b1 -= lr * db1
91     W2 -= lr * dW2; b2 -= lr * db2
92     return W1, b1, W2, b2
93
94 def make_nn(input_dim, output_dim, hidden=32):
95     W1 = np.random.randn(input_dim, hidden) * np.sqrt(2/input_dim
96     )
97     b1 = np.zeros(hidden)
98     W2 = np.random.randn(hidden, output_dim) * np.sqrt(2/hidden)
99     b2 = np.zeros(output_dim)
    return W1, b1, W2, b2
```

Listing A.1: Complete Training.py

Appendix B

Full Annotated Code: pipeline.py

```
1 # MODEL LOADER
2 def load_model(depth):
3     path = MODEL_FILES[depth]
4     if not os.path.exists(path):
5         raise FileNotFoundError(...)
6     m = np.load(path)
7     return m['W1'], m['b1'], m['W2'], m['b2']
8
9 # INFERENCE
10 def nn_forward(z, W1, b1, W2, b2):
11     h = np.maximum(0, z @ W1 + b1)
12     return np.tanh(h @ W2 + b2) * np.pi
13
14 def generate_params(W1, b1, W2, b2, n=1, seed=None):
15     rng = np.random.default_rng(seed)
16     return [nn_forward(rng.standard_normal(W1.shape[0]),
17                       W1, b1, W2, b2) for _ in range(n)]
18
19 # CLI
20 if __name__ == '__main__':
21     parser = argparse.ArgumentParser(...)
22     parser.add_argument('--pipeline', choices=[
23         'vqe', 'benchmark', 'paramgen', 'export', 'all'])
24     args = parser.parse_args()
25     if args.pipeline in ('vqe', 'all'):
26         pipeline_vqe(args.depth, ...)
27     # ... etc
```

Listing B.1: Key sections of pipeline.py (abbreviated for clarity)

Appendix C

Mathematical Reference

C.1 Pauli Algebra

The Pauli matrices satisfy the commutation relations:

$$[X, Y] = 2iZ, \quad [Y, Z] = 2iX, \quad [Z, X] = 2iY. \quad (\text{C.1})$$

And anti-commutation relations: $\{X, Y\} = 0$, etc. They are also self-inverse: $X^2 = Y^2 = Z^2 = \mathbb{I}$.

C.2 Matrix Exponential for Rotations

Using the identity $e^{i\phi\hat{n}\cdot\boldsymbol{\sigma}/2} = \cos(\phi/2)\mathbb{I} + i\sin(\phi/2)(\hat{n}\cdot\boldsymbol{\sigma})$:

$$R_y(\theta) = e^{-i\theta Y/2} = \cos \frac{\theta}{2} \mathbb{I} - i \sin \frac{\theta}{2} Y = \begin{pmatrix} \cos \frac{\theta}{2} & -\sin \frac{\theta}{2} \\ \sin \frac{\theta}{2} & \cos \frac{\theta}{2} \end{pmatrix}, \quad (\text{C.2})$$

$$R_z(\theta) = e^{-i\theta Z/2} = \cos \frac{\theta}{2} \mathbb{I} - i \sin \frac{\theta}{2} Z = \begin{pmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{pmatrix}. \quad (\text{C.3})$$

C.3 Derivation of the Parameter-Shift Rule

Consider $C(\theta) = \langle \psi_0 | e^{i\theta G} \hat{O} e^{-i\theta G} | \psi_0 \rangle$ where G has eigenvalues $\pm \frac{1}{2}$. Then:

$$\frac{dC}{d\theta} = i \langle [G, \hat{O}] \rangle = \frac{1}{2} \left[C\left(\theta + \frac{\pi}{2}\right) - C\left(\theta - \frac{\pi}{2}\right) \right]. \quad (\text{C.4})$$

The second equality uses the fact that for a generator with eigenvalues $\pm \frac{1}{2}$, the spectrum of $e^{i\frac{\pi}{2}G}$ is $\{e^{\pm i\pi/4}\}$, and the expectation value difference exactly captures the derivative.

C.4 Adam Convergence Properties

Adam converges to a stationary point under mild conditions (bounded gradients, non-stationarity of the landscape). The bias-corrected moment estimates $\hat{m}_t$ and $\hat{v}_t$ converge to the true first and second moments of the gradient distribution as $t \rightarrow \infty$:

$$\hat{m}_t \xrightarrow{t \rightarrow \infty} \mathbb{E}[g], \quad \hat{v}_t \xrightarrow{t \rightarrow \infty} \mathbb{E}[g^2]. \quad (\text{C.5})$$

The effective step size is $\alpha \hat{m}_t / \sqrt{\hat{v}_t} \approx \alpha \text{sign}(g)$ when g is consistently signed, and decreases as $\alpha/\sqrt{t}$ when gradients are noisy.

Bibliography

- [1] L. Friedrich and J. Maziero, *Avoiding barren plateaus with classical deep neural networks*, Phys. Rev. A **106**, 042433 (2022). <https://arxiv.org/abs/2205.13418>
- [2] J. R. McClean, S. Boixo, V. N. Smelyanskiy, R. Babbush, and H. Neven, *Barren plateaus in quantum neural network training landscapes*, Nature Communications **9**, 4812 (2018). <https://arxiv.org/abs/1803.11173>
- [3] M. Cerezo, A. Sone, T. Volkoff, L. Cincio, and P. J. Coles, *Cost function dependent barren plateaus in shallow parametrized quantum circuits*, Nature Communications **12**, 1791 (2021). <https://arxiv.org/abs/2001.00550>
- [4] A. Peruzzo et al., *A variational eigenvalue solver on a photonic quantum processor*, Nature Communications **5**, 4213 (2014).
- [5] D. P. Kingma and J. Ba, *Adam: A method for stochastic optimization*, Proc. ICLR (2015). <https://arxiv.org/abs/1412.6980>
- [6] K. Mitarai, M. Negoro, M. Kitagawa, and K. Fujii, *Quantum circuit learning*, Phys. Rev. A **98**, 032309 (2018).
- [7] M. Schuld, V. Bergholm, C. Gogolin, J. Izaac, and N. Killoran, *Evaluating analytic gradients on quantum hardware*, Phys. Rev. A **99**, 032331 (2019).
- [8] K. He, X. Zhang, S. Ren, and J. Sun, *Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification*, Proc. ICCV (2015).
- [9] Qiskit contributors, *Qiskit: An Open-source Framework for Quantum Computing*, (2023). <https://qiskit.org>
- [10] S. Sachdev, *Quantum Phase Transitions*, 2nd ed. (Cambridge University Press, 2011).
- [11] J. Preskill, *Quantum Computing in the NISQ Era and Beyond*, Quantum **2**, 79 (2018).
- [12] M. Cerezo et al., *Variational quantum algorithms*, Nature Reviews Physics **3**, 625–644 (2021).